TITAN SYSTEM

Ultimate Cursor Implementation Blueprint

Backend: Node.js + TypeScript

Versions Covered: v1 → v6+

1. PROJECT STRUCTURE

```
/titan-system
 ■■■ src
 ■   ■■■ core
 ■   ■■■ modules
 ■   ■    ■■■ auth
 ■   ■    ■■■ users
 ■   ■    ■■■ crm
 ■   ■    ■■■ contracts
 ■   ■    ■■■ billing
 ■   ■    ■■■ analytics
 ■   ■    ■■■ phase-launch
 ■   ■■■ database
 ■   ■■■ config
 ■   ■■■ api
 ■   ■■■ tests
 ■■■ docker-compose.yml
 ■■■ Dockerfile
 ■■■ tsconfig.json
 ■■■ package.json
```

Strict modular architecture. No cross-module logic leakage allowed.

2. TITAN CORE ENGINE

```
// core/CoreEngine.ts
export class CoreEngine {
  constructor(private registry: ModuleRegistry) {}
  async bootstrap() {
    await this.registry.loadModules();
  }
}
```

```
// core/ModuleRegistry.ts
export class ModuleRegistry {
  private modules = [];
  register(module) { this.modules.push(module); }
  async loadModules() {
    for (const m of this.modules) await m.init();
  }
}
```

Core controls lifecycle, dependency injection and module boot order.

3. MODULE STRUCTURE (Example: Contracts)

```
contracts/
 ■■■ contracts.controller.ts
 ■■■ contracts.service.ts
 ■■■ contracts.repository.ts
 ■■■ contracts.dto.ts
 ■■■ contracts.module.ts
 ■■■ contracts.test.ts
```

Controller = HTTP layer. Service = business logic. Repository = DB only.

4. DATABASE SCHEMA (PostgreSQL)

```
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE contracts (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  status VARCHAR(50),
  value NUMERIC,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE invoices (
  id UUID PRIMARY KEY,
  contract_id UUID REFERENCES contracts(id),
  amount NUMERIC,
  status VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW()
);
```

All access via repository layer. Migrations via migration manager only.

5. TESTING STRATEGY

- Unit test for every service
- Integration test for contract → invoice flow
- System test for full lifecycle
- Minimum 90% coverage required
- Failing test blocks migration

6. MIGRATION PROTOCOL

- Run full test suite
- Backup database
- Run migration scripts
- Validate checksum
- Promote test-main → master-main
- Enable phase flags

7. PHASE LAUNCH SYSTEM

```
if (process.env.PHASE === 'v3') {
   enableModule('billing');
}
```

Feature flags must be centrally managed.

8. DEPLOYMENT

```
# Dockerfile
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
CMD ["node", "dist/main.js"]
```

Production requires Docker, environment separation and monitoring.

9. VERSION EVOLUTION v1 → v6+

- v1: Core + CRM + Contracts + Billing
- v2: Auth + Logging + Environment separation
- v3: Payment integration + Scheduler
- v4: API Gateway + Cache + Backup
- v5: Security hardening + GDPR + CI/CD
- v6: Master control panel + Optimization
- v6+: Microservices + Horizontal scaling + AI layer